

OfferHub

Mikroservis Mimarisi ve Docker
Kapsamlı Teknik Rehber

Backend Ekibi için Sıfırdan, Örnekli Anlatım

Java Spring · Docker Compose · Event-Driven Mimari · Flutter Entegrasyonu

Ağustos 2026

İçindekiler

1. Neden Mikroservis? Monolit ile Karşılaştırma
2. Docker'ın Temelleri
3. Docker Compose ile Çoklu Servis Orkestrasyonu
4. OfferHub Genel Mimarisi
5. Database-per-Service Prensipleri
6. API Gateway
7. Servisler Arası İletişim: REST ve Event
8. Dört Servisin Detaylı Görevleri
9. Mobil – Backend Haberleşmesi
10. Lokal Geliştirme ve Telefon Bağlantısı
11. Bağımsızlık ve Dayanıklılık Demosu
12. Güvenlik Gereksinimleri Özeti
13. Sözlük

BÖLÜM 0

Bu Doküman Hakkında

Bu rehber, OfferHub staj projesine kod yazmaya başlamadan önce mimarinin bütününe kafanda netleştirmen için hazırlandı. Amaç, Docker'ın ve mikroservis mimarisinin "neden" böyle kurgulandığını, servislerin birbiriyle nasıl konuştuğunu ve mobil uygulamanın backend'e nasıl bağlandığını, doğrudan sizin projenizden alınan somut örneklerle anlatmak. Genel geçer teoriden değil, OfferHub'ın kendi senaryolarından (kampanya oluşturma, AI skorlama, rozet kazanma, servis kapatma demosu) yola çıkıyoruz.

Doküman sırayla okunacak şekilde kurgulandı: önce mimari felsefe (neden mikroservis), sonra araç (Docker), sonra sizin projenizin somut yapısı. Sonunda kullanacağın terimlerin kısa bir sözlüğü var — takıldığın bir terimi hızlıca hatırlamak için oraya bakabilirsin.

BÖLÜM 1

Neden Mikroservis? Monolit ile Karşılaştırma

Normal bir okul/staj projesinde genelde tek bir Spring Boot uygulaması yazılır: tek **pom.xml**, tek veritabanı, tek **java -jar** ile ayağa kalkan tek bir process. Buna **monolit** mimari denir. OfferHub'da Identity, Campaign, AI ve Gamification hepsi aynı kod tabanında, aynı process içinde olsaydı, bu bir monolit olurdu.

Case dokümanı bunu bilerek yasaklıyor ve mikroservis mimarisi zorunlu tutuyor. Sebebi gerçek dünya senaryosuyla doğrudan ilgili: bir telekom operatörünün sistemi tek process'te olamaz. AI modeli çıktığında kampanya oluşturmanın da çökmesi kabul edilemez bir risktir. Mikroservis mimarisinde her iş alanı (Identity, Campaign, AI, Gamification) kendi başına çalışabilen, kendi başına deploy edilebilen, kendi veritabanına sahip **ayrı bir uygulama** olur. Biri arızalanırsa diğerleri yaşamaya devam eder.

Neden önemli

Case'in kapanış demosundaki en kritik adım — "bir servisi docker stop ile kapat, sistemin geri kalanının çalıştığını kanıtla" — tam olarak bu mimari kararın sonucudur. Mikroservis mimarisi doğru kurulmadıysa bu demo başarısız olur.

Özellik	Monolit	Mikroservis (OfferHub)
Kod tabanı	Tek proje, tek repo klasörü	Her serviste ayrı proje (backend/identity-service, campaign-service, ...)
Veritabanı	Tek ortak veritabanı	Servis başına ayrı veritabanı (database-per-service)
Deploy	Tek seferde her şey	Her servis bağımsız derlenir, bağımsız container'da çalışır
Arıza etkisi	Bir hata tüm sistemi düşürebilir	Bir servis çökerse diğerleri çalışmaya devam eder
İletişim	Doğrudan metod çağırısı (aynı process)	Ağ üzerinden REST çağırısı veya event/mesaj kuyruğu
Bedeli	Basit ama kırılgan	Daha karmaşık (ağ, senkronizasyon) ama dayanıklı

Bedelini de görmek gerekir: artık 4 ayrı Spring Boot uygulaması, 4 ayrı veritabanı var ve bunların birbiriyle **ağ üzerinden** konuşması gerekiyor. Aynı process içinde bir metod çağırmak yerine, bir servisten diğerine HTTP isteği atılıyor ya da bir mesaj kuyruğuna event bırakılıyor. Bu karmaşıklığı yönetilebilir kılmak için de Docker ve Docker Compose devreye giriyor — sıradaki iki bölüm tam olarak bunu anlatıyor.

BÖLÜM 2

Docker'ın Temelleri

2.1 Container nedir, VM'den farkı ne

Docker container'ı "hafif bir sanal makine" gibi düşünmek yaygın bir yanılgıdır. Bir sanal makine (VM), üzerinde tam bir işletim sistemi çalıştırır: kendi çekirdeği (kernel), GB'larca boyut, dakikalar süren açılış süresi. Container ise host bilgisayarın çekirdeğini paylaşır; sadece uygulamayı ve onun bağımlılıklarını (Java runtime, kütüphaneler, konfigürasyon dosyaları) izole bir kutuya koyar. Saniyeler içinde açılır, çok daha az kaynak (RAM/disk) tüketir.

Kriter	Sanal Makine (VM)	Container (Docker)
Boyut	Genelde GB'larca (tam işletim sistemi)	Genelde MB'larca (sadece uygulama + bağımlılıklar)
Başlama süresi	Dakikalar	Saniyeler
İzolasyon	Tam donanım seviyesinde izolasyon	Process seviyesinde izolasyon, kernel paylaşımlı
Taşınabilirlik	Görece ağır, taşınması zor	Image olarak paketlenip her yerde aynı çalışır

2.2 Temel kavramlar

- **Dockerfile** — Bir servisi nasıl çalıştıracakını anlatan tarif dosyası ("Java 21 runtime'ı al, benim derlediğim jar'ı içine koy, java -jar app.jar ile başlat" gibi adımlar).
- **Image** — Dockerfile'in **docker build** ile derlenmiş hâli; değişmez bir kalıp (şablon).
- **Container** — Bir image'dan **docker run** ile başlatılmış, çalışan kopya. Aynı image'dan istenildiği kadar container başlatılabilir, hepsi birbirinden izole çalışır.
- **Registry** — Image'ların depolandığı yer (örn. Docker Hub). **postgres:16** gibi hazır image'lar buradan indirilir.

Örnek — identity-service için Dockerfile

```
# backend/identity-service/Dockerfile

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/identity-service.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Bu tarif: Java 21 çalışma zamanını temel al, derlenmiş jar'ı kopyala, 8080 portunu aç, başlarken jar'ı çalıştır.

Bu bilgiyle artık her servisin kendi ortamının tamamen izole ve tekrarlanabilir olduğunu görüyorsunuz: sen kendi laptopunda identity-service'i Docker ile ayağa kaldırdığında, takım

arkadaşın kendi laptopunda aynı image'ı ayağa kaldırdığında, ikisi de birebir aynı ortamda çalışır — "benim bilgisayarımda çalışıyordu" sorunu ortadan kalkar.

BÖLÜM 3

Docker Compose ile Çoklu Servis Orkestrasyonu

Tek bir servisi Docker ile çalıştırmak kolay, ama OfferHub'da 4 servis + 4 veritabanı + 1 gateway var — yani 9 ayrı container. Her birini tek tek **docker run** ile elle başlatmak, aralarındaki ağ bağlantısını elle kurmak son derece hataya açık olurdu. **Docker Compose**, bunların hepsini tek bir **docker-compose.yml** dosyasında tarif etmeni ve **docker compose up** tek komutuyla hepsini birden, doğru sırada ve birbirini bulacak şekilde ayağa kaldırmanı sağlar.

3.1 Örnek docker-compose.yml (özet)

```
services:
  identity-service:
    build: ./backend/identity-service
    ports:
      - "8081:8080"
    environment:
      - DB_URL=jdbc:postgresql://identity-db:5432/identity
    depends_on:
      - identity-db

  identity-db:
    image: postgres:16
    environment:
      - POSTGRES_DB=identity
    volumes:
      - identity-data:/var/lib/postgresql/data

  campaign-service:
    build: ./backend/campaign-service
    ports:
      - "8082:8080"
    environment:
      - DB_URL=jdbc:postgresql://campaign-db:5432/campaign
    depends_on:
      - campaign-db

  api-gateway:
    build: ./backend/api-gateway
    ports:
      - "8080:8080"
    depends_on:
      - identity-service
      - campaign-service
      - ai-service
      - gamification-service

volumes:
  identity-data:
```

Gerçek dosyada campaign-db, ai-service, ai-db, gamification-service, gamification-db bölümleri de aynı desende eklenir.

3.2 Servisler birbirini nasıl bulur? (Dahili DNS)

Dikkat edilmesi gereken kritik nokta: **identity-service**, veritabanına bağlanırken **localhost:5432** değil, **identity-db:5432** diye bağlanıyor. Çünkü Compose, aynı dosyada tanımlı tüm servisleri otomatik olarak aynı sanal ağa koyar ve her birine servis adını bir **DNS ismi** olarak verir. **identity-service** container'ının içinden **identity-db** diye bir hostname'e istek atarsan, Docker bunu otomatik olarak doğru container'a yönlendirir.

Pratik sonuç

Bu mekanizma, database-per-service kuralını fiziksel olarak da uygulanır hale getiriyor: campaign-service, aynı ağda olsa bile identity-db'nin adını bilmediği ve erişim izni olmadığı için, Identity'nin veritabanına asla direkt SQL atamaz.

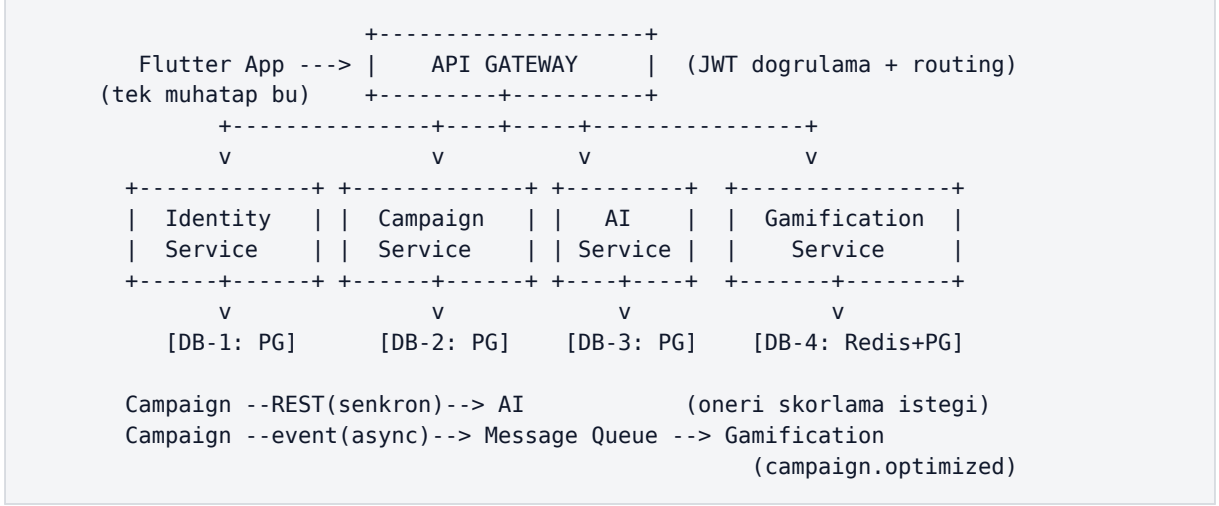
3.3 Sık kullanılacak komutlar

Komut	Ne yapar
docker compose up	compose.yml'deki tüm servisleri build edip başlatır
docker compose up -d	Aynısını arka planda (detached) çalıştırır
docker compose down	Tüm container'ları durdurur ve kaldırır
docker compose logs -f identity-service	Sadece bir servisin canlı loglarını izler
docker stop identity-service	Demoda "bir servisi kapat" adımı için — sadece o servisi durdurur
docker compose ps	Hangi servislerin ayakta/sağlıklı olduğunu listeler

BÖLÜM 4

OfferHub Genel Mimarisi

Aşağıdaki şema, case dokümanındaki mimariyi birebir yansıtıyor. Mobil uygulama tek bir kapıdan (API Gateway) içeri girer; Gateway isteği ilgili servise yönlendirir; her servisin kendi, birbirine kapalı veritabanı vardır.



OfferHub bileşen diyagramı: mobil, gateway, 4 servis, 4 veritabanı ve aralarındaki iki farklı iletişim türü.

4.1 Bileşenlerin bir cümlelik özeti

Bileşen	Görevi
Flutter App	Abone, uzman, süpervizör, admin ekranları. Sadece Gateway ile konuşur.
API Gateway	Tek giriş noktası: routing, JWT doğrulama, rate limiting.
Identity Service	Kayıt, giriş, JWT üretimi/yenilemesi, rol/yetki, audit log.
Campaign Service	Kampanya yaşam döngüsü, state machine, SLA takibi, segment yönetimi.
AI Service	Öneri skortlama, segment sınıflandırma, akıllı uzman ataması.
Gamification Service	Event dinleyerek puan, rozet, seviye, liderlik tablosu hesaplar.

BÖLÜM 5

Database-per-Service Prensipleri

Bir düşünce deneyi: eğer campaign-service, doğrudan identity-db'ye SQL sorgusu atabilseydi, iki servis arasında gizli, görünmez bir bağımlılık oluşurdu. Identity ekibi kendi tablo şemasını değiştirdiğinde, Campaign servisi habersizce bozulurdu. Bu da mikroservisin bütün amacını — bağımsız deploy edilebilirlik — ortadan kaldırırdı.

Bunun yerine kural nettir: bir servis, başka bir servisin verisine ihtiyaç duyduğunda onun veritabanına girmez; onun **API'sine HTTP isteği atar** ya da onun **yayınladığı event'leri dinler**. Veritabanı sınırı, aynı zamanda servis sınırıdır.

Servis	Önerilen veritabanı	Neden
Identity Service	PostgreSQL	İlişkisel veri: kullanıcılar, roller, refresh token'lar, audit log
Campaign Service	PostgreSQL	Kampanya, vaka, state machine geçişleri — ilişkisel ve tutarlılık kritik
AI Service	PostgreSQL (+ eğitim verisi dosya/CSV olarak)	Skor/tahmin kayıtları, doğruluk takibi
Gamification Service	Redis + PostgreSQL	Redis: anlık liderlik tablosu (hızlı okuma). PostgreSQL: kalıcı puan/rozet geçmişi

Unutma

"Database-per-service" sadece mantıksal bir kural değil — Bölüm 3.2'de gördüğün Docker network izolasyonu sayesinde bu kural fiziksel olarak da uygulanmış olur. Bir servis, başka servisin DB'sinin hostname'ini bile bilmez.

BÖLÜM 6

API Gateway

Case dokümanında net bir kural var: "Mobil uygulama yalnızca API Gateway ile konuşur; servislere doğrudan istek atmaz." Bunun sebebi basit: mobil uygulama 4 farklı servisin adresini, portunu bilmek zorunda kalmayın. Gateway, dışarıya **tek bir kapı** gösterir (local geliştirmede **localhost:8080**) ve gelen isteği path'e (URL yoluna) göre doğru servise yönlendirir.

6.1 Routing örneği

```
/api/v1/auth/**      -> Identity Service
/api/v1/campaigns/** -> Campaign Service
/api/v1/ai/**        -> AI Service
/api/v1/game/**      -> Gamification Service

Örnek:
POST /api/v1/auth/login      -> identity-service:8080/login
POST /api/v1/campaigns      -> campaign-service:8080/campaigns
GET  /api/v1/game/leaderboard -> gamification-service:8080/leaderboard
```

6.2 Gateway'in ikinci büyük görevi: JWT doğrulama

Mobil uygulama her istekte **Authorization: Bearer <token>** header'ı gönderir. Gateway, isteği ilgili servise yönlendirmeden önce bu token'ın geçerli olup olmadığını kontrol eder (imza doğru mu, süresi dolmuş mu). Böylece 4 servisin her biri kendi içinde ayrı ayrı "bu token geçerli mi" kodu yazmak zorunda kalmaz — merkezi bir yerde yapılır.

Rol bazlı yetki kontrolü (örneğin "abone token'ıyla süpervizör endpoint'ine erişilemez") genelde iki katmanda uygulanır: Gateway'de kaba bir ön kontrol, servisin kendi içinde endpoint seviyesinde ince taneli kontrol. Güvenlik testlerinde mentörler ikisini de deneyecek, bu yüzden tek katmana güvenmemek gerekiyor.

6.3 Teknoloji seçenekleri

- **Spring Cloud Gateway** — Java/Spring ekosistemine en doğal uyan, tercih edilmesi mantıklı seçenek.
- **Kong / Ocelot / Express Gateway** — Hazır, dil bağımsız alternatifler.
- **Kendi yazdığınız reverse proxy** — Case bunu da kabul ediyor, ama zaman kısıtı düşünülünce hazır bir çözüm daha gerçekçi.

BÖLÜM 7

Servisler Arası İletişim: REST ve Event

Projede iki farklı iletişim şekli var, ikisini de net ayırt etmek gerekiyor.

7.1 Senkron iletişim — REST çağrısı

Campaign Service, bir kampanya oluşturulduğunda "bu abone için öneri skoru nedir" diye AI Service'e doğrudan HTTP isteği atar ve **cevabı bekler**:

```
POST /api/v1/ai/recommend (Campaign -> AI)
{
  "subscriber_id": "a1b2c3",
  "campaign_type": "EK_PAKET"
}

Cevap:
{
  "score": 0.83,
  "conversion_probability": 0.61,
  "segment": "YUKSEK_DEGER"
}
```

Bu senkron bir çağrıdır çünkü Campaign, cevap gelene kadar bekler ve kampanyayı ona göre kaydeder. Sorun şu: AI Service çökmüşse bu çağrı timeout olur. Case, bunun için bir fallback istiyor: AI cevap vermezse Campaign, segment'i **BELIRSIZ** yaparak kampanyayı yine de kaydetmeli, öncelik **ORTA** verip manuel kuyruğa düşürmeli. Bunu kodda genelde try-catch + timeout + devre kesici (circuit breaker, örn. Resilience4j) ile yaparsınız.

7.2 Asenkron iletişim — Event / Mesaj Kuyruğu

Gamification Service'in puan vermesi için Campaign Service'in Gamification'ı **beklemesi** hiç mantıklı değil — puan hemen hesaplanmasa da kullanıcı deneyimini etkilemez. Bunun yerine Campaign, bir uzman optimizasyonu tamamladığında mesaj kuyruğuna (RabbitMQ / Redis pub-sub / Kafka) bir mesaj **bırakır** ve hemen devam eder:

```
// Campaign Service bunu kuyruğa YAYINLAR (publish), kimseyi beklemez
{
  "event_type": "campaign.optimized",
  "timestamp": "2026-08-17T14:22:10Z",
  "payload": {
    "case_id": "CMP-2026-000123",
    "expert_id": "a7f3...",
    "segment": "RISKLI_KAYIP",
    "conversion_lift": 0.18
  }
}
```

Gamification Service arka planda bu kuyruğu **dinler** (subscribe), mesaj geldiğinde kendi mantığını çalıştırır: süreyi hesaplar, +10 puan ekler (hedef aşıldıysa +15), rozet koşullarını kontrol eder, gerekirse **badge.earned** event'i üretir. Campaign Service'in Gamification'dan haberi bile yoktur — sadece "böyle bir şey oldu" diye bağırır, kim dinlerse dinler.

Analoji

REST çağırısı = telefonla arayıp cevap bekleme. Event = birine mesaj atıp meşgulse bile devam etme; mesaj onda duruyor, o ne zaman bakarsa okuyor. Bu farkı net kavramak, hangi durumda hangisini seçeceğinizi otomatik olarak belirler.

7.3 Ne zaman hangisi?

Durum	Tercih	Neden
Kampanya oluşturulurken AI'dan skor isteme	REST (senkron)	Kampanya kaydı, skor bilgisine ihtiyaç duyar; cevap hemen lazım
Optimizasyon tamamlanınca puan/rozet hesaplama	Event (asenkron)	Puanın hemen hesaplanması şart değil; Campaign, Gamification'ı beklememeli
Segment override → AI doğruluk takibi	Event (asenkron)	Kayıt amaçlı bildirim; anlık cevap gerekmiyor
SLA aşımı bildirimi	Event (asenkron)	Arka planda tetiklenen, bekletici olmayan bir sonuç

BÖLÜM 8

Dört Servisin Detaylı Görevleri

8.1 Identity Service

Sistemin kimlik ve yetki omurgası. Abone kaydı (GSM + sabit OTP **1234** simülasyonu), personel hesapları (admin tarafından oluşturulur), şifre politikası (bcrypt/argon2 hash), hesap kilitleme (5 başarısız girişte 15 dk kilit) burada yönetilir.

- **Access token:** JWT, 15 dakika ömürlü. Payload'ında user_id, rol, uzmanlık/bölge bilgisi taşır.
- **Refresh token:** 7 gün ömürlü, veritabanında saklanır.
- **Token rotation:** Refresh token kullanıldığında yenisi üretilir, eskisi geçersiz kılınır. Geçersiz kılınmış bir refresh token tekrar kullanılmaya çalışılırsa (token theft belirtisi), o kullanıcının **tüm oturumları** sonlandırılır.
- **Rol/yetki matrisi:** her endpoint seviyesinde uygulanır; yetkisiz erişim 403 döner ve audit log'a yazılır.
- **Audit log:** kim, ne, ne zaman, nereden, hangi sonuçla — giriş denemeleri, kilitlemeler, rol değişiklikleri, yetkisiz erişim denemeleri burada kayıt altına alınır.

8.2 Campaign Service

Kampanya yaşam döngüsünün ve optimizasyon vakalarının kalbi.

- Kampanya oluşturma: başlık, tip, hedef segment, indirim oranı; benzersiz kampanya numarası (CMP-2026-000123).
- Kampanya bir segmente hedeflendiğinde otomatik olarak AI Service'e gönderilir (Bölüm 7.1'deki REST çağırısı).
- **State machine:** YENİ → ATANDI → OPTIMIZE_EDILIYOR → TEST_EDILIYOR → TAMAMLANDI → YAYINDA → ARSIVLENDI. Kural dışı bir geçiş denemesi (örn. YENI'den direkt TAMAMLANDI'ya) **422** hatası döner.
- **SLA takibi:** öncelik seviyesine göre süre sayılır (KRITIK: 2 saat, YUKSEK: 8 saat, ORTA: 24 saat, DUSUK: 72 saat). Demo kolaylığı için SLA_TIME_UNIT env değişkeniyle bu süreler dakikaya kısaltılabilir yapılmalı.

8.3 AI Service

Projenin kalbi olarak tanımlanıyor, çünkü mock/hardcoded olması kesinlikle kabul edilmiyor — girdi değişince çıktı da değişmeli. Üç görevi var:

- **Öneri skortlama:** abone profiline göre her kampanya için 0.0-1.0 arası skor + dönüşüm olasılığı üretir. 0.60 altı skorlar aboneye hiç gösterilmez, 0.80 üstü öncelikli gösterilir.
- **Segment sınıflandırma:** YUKSEK_DEGER, RISKLI_KAYIP, YENI_ABONE, PASIF etiketlerinden birini atar.
- **Akıllı uzman ataması:** $\text{skor} = (\text{uzmanlık_eslesme} \times 0.5) + (\text{bosluk_orani} \times 0.3) + (\text{performans} \times 0.2)$ formülüyle en uygun uzmana otomatik atama yapar.

Yaklaşım serbest: kendi eğittiğiniz klasik ML modeli, kural tabanlı + ML hibrit, ya da LLM API entegrasyonu olabilir. Hangisini seçerseniz seçin, AI yaklaşım dokümanında (README dışı ayrı bir doküman) nasıl çalıştığını anlatmanız bekleniyor.

8.4 Gamification Service

Bu servisin en ayırt edici özelliği: doğrudan çağrılmaz, tamamen **event tabanlı** çalışır (Bölüm 7.2). Campaign Service'ten gelen event'leri dinleyerek puan tablosunu işletir (+10 optimizasyon tamamlama, +5 hızlı bonus, +15 hedef aşımı, -5 SLA aşımı), rozet koşullarını kontrol eder (İlk Kampanya, Hız Ustası, Churn Avcısı, vb.), seviye hesaplar (Bronz/Gümüş/Altın/Platin) ve günlük/haftalık liderlik tablosunu üretir.

BÖLÜM 9

Mobil - Backend Haberleşmesi

9.1 Tek kapı kuralı

Mobil uygulama tek bir base URL bilir: Gateway'in adresi. Tüm istekler bu tek adrese gider, path farkına göre Gateway hangi servise yönlendireceğine karar verir. Mobil tarafta bu genelde tek bir API client (Flutter'da **Dio** paketiyle) olarak kurulur.

9.2 Standart response zarfı

```
Basarili:
{ "success": true, "data": { ... }, "error": null }

Hatali:
{ "success": false, "data": null,
  "error": { "code": "INVALID_CREDENTIALS", "message": "..." } }
```

Bu zarf Gün 2'deki API sözleşmesi toplantısında kilitlenmeli. Aksi hâlde her ekran farklı hata formatı parse etmeye çalışır, mobil tarafta ciddi karmaşa yaratır.

9.3 Auth akışı

- Abone: GSM + OTP (**1234** sabit kod) ile **POST /api/v1/auth/otp-verify**. Personel: e-posta + şifre ile **POST /api/v1/auth/login**.
- Cevapta `access_token` (15 dk) ve `refresh_token` (7 gün) döner; mobil bunları güvenli depolamada saklar (Flutter'da `flutter_secure_storage`).
- **Şeffaf token yenileme**: mobil tarafta bir HTTP interceptor kurulur. İstek 401 dönerse, interceptor otomatik olarak `/auth/refresh` çağırır, yeni token'ları alır, orijinal isteği sessizce tekrar dener. Kullanıcı bunu hiç fark etmez.
- Refresh token rotation nedeniyle her yenilemede alınan yeni `refresh_token` da güncellenip saklanmalı.
- JWT payload'ındaki rol bilgisi decode edilerek **rol bazlı navigasyon** kurulur — abone hesabıyla girmiş biri süpervizör ekranını hiç göremez.

9.4 Mock-önce, gerçek-sonra iş akışı

Backend Gün 1-2'de henüz kod yazmamışken mobil ekibin boş oturmaması için: Gün 2'de üç kişi birlikte API sözleşmesini kilitler; mobil bu sözleşmeye göre Postman Mock Server / MSW / json-server ile sahte bir API kurup Hafta 1 boyunca UI'ı geliştirir. Backend hazır oldukça (Identity ~Gün 8-9 civarı) mobil, mock base URL'i gerçek Gateway URL'iyle değiştirir — sözleşme doğru yazıldıysa kod tarafında neredeyse hiçbir şey değişmez.

9.5 Dokümantasyon ve gerçek zamanlılık

En az Campaign ve AI servisleri için Swagger/OpenAPI dokümantasyonu zorunlu — mobil geliştirici endpoint'leri elle sormak yerine **/swagger-ui.html** üzerinden görüp dener. Rozet bildirimi ve liderlik tablosu gibi yerlerde "gerçek zamanlı" kelimesi geçse de case, alternatif

olarak "sayfa yenilemede güncel" olmayı da kabul ediyor — yani WebSocket kurmaya gerek yok, ilgili ekran her açıldığında/yenilendiğinde endpoint tekrar çağrılabilir.

BÖLÜM 10

Lokal Geliştirme ve Telefon Bağlantısı

"Telefon ile backend'i haberleştirmek için bulut sunucu mu gerekiyor?" sorusunun cevabı hayır. Geliştirme ve hatta kapanış demosu için bile lokal ağ yeterli.

10.1 Emulator / Simulator kullanıyorsanız

- **Android emulator:** laptop'taki localhost'a **10.0.2.2** adresinden ulaşılır (emulator'un kendi sanal ağı bu özel adresi host makineye yönlendirir). Flutter base URL: **http://10.0.2.2:8080**
- **iOS Simulator:** host makineyle ağı paylaşır, doğrudan **http://localhost:8080** çalışır.

10.2 Gerçek telefonla test/demo ederken

Telefon ile laptop **aynı Wi-Fi ağı**nda olmalı. Adımlar:

- Laptop'un yerel IP'sini bulun: **ipconfig** (Windows) → Wi-Fi adaptöründeki IPv4 Address (örn. 192.168.1.23).
- Flutter'da base URL'i bu IP'ye çevirin: **http://192.168.1.23:8080**.
- Windows Firewall'da ilgili portun (8080) gelen bağlantılara açık olduğundan emin olun.
- Docker Compose'daki port eşlemesi ("**8080:8080**") varsayılan olarak tüm arayüzlerden (0.0.0.0) dinler, yani ekstra bir ayar gerekmez.

Pratik ipucu

Flutter'da base URL'i sabit kodlamak yerine bir config/env değişkeni yapın (--dart-define=API_BASE_URL=...). Böylece emulator'da 10.0.2.2, demo günü laptop IP'si arasında geçiş yaparken kod değiştirmeniz gerekmez.

10.3 Bulut sunucu gerekli mi?

Case'in kabul kriterlerinde ve kapanış demosunda (docker compose up ile canlı, lokal demo) bulut deploy zorunluluğu yok. Teslimatta "varsa APK/TestFlight bağlantısı" deniyor — yani opsiyonel. 20 günlük süre düşünüldüğünde öncelik lokal ağ üzerinden çalışır hale getirmek olmalı; bulut deploy zaman kalırsa hafta 4'te bir ekstra olarak değerlendirilebilir.

BÖLÜM 11

Bağımsızlık ve Dayanıklılık Demosu

Case'teki en kritik kapanış anı: bir servisi **docker stop** ile kapatıp sistemin geri kalanının çalıştığını göstermek. Bu, Bölüm 5 (ayrı DB) ve Bölüm 7.2 (event tabanlı iletişim) kararlarının doğrudan meyvesidir.

Senaryo	Ne olur
AI Service kapatılırsa	Campaign, AI'a REST çağrısı atar, cevap gelmez, timeout'a düşer. Segment BELIRSIZ, öncelik ORTA olarak kampanya yine kaydedilir; kampanya oluşturma çökmez.
Gamification Service kapatılırsa	Campaign yine campaign.optimized event'ini kuyruğa atar. Kimse anlık dinlemese de mesaj kuyrukta bekler; Gamification geri geldiğinde işler.
Identity Service kapatılırsa	Zaten aktif olan JWT'ler geçerliliğini korur (Gateway kendi başına imza doğrulayabilir); ama yeni giriş/refresh işlemleri o an yapılamaz.

Eğer bunun yerine her şey senkron REST ile ve ortak bir veritabanıyla yapılsaydı, bir servis çöktüğünde domino etkisiyle hepsi çökerdi — mikroservis olmanın hiçbir anlamı kalmazdı. Bu yüzden bu demo adımı, mimarinizin gerçekten doğru kurulup kurulmadığının somut kanıtıdır; case de bu adım için önceden prova yapmanızı özellikle vurguluyor.

BÖLÜM 12

Güvenlik Gereksinimleri Özeti

Kapanış demosunda mentörler kasıtlı saldırı senaryoları deneyecek. Amaç "yakalanma" değil, güvenli kod yazma pratiği — hangi açığın neden oluştuğunu anlatabilmeniz de sonuç kadar önemli.

Senaryo	Ne test ediliyor	Temel önlem
SQL Injection	Form alanına ' OR 1=1 -- gibi girdi	ORM/prepared statement kullanmak, ham SQL string birleştirmemek
Yetkisiz endpoint erişimi	Abone token'ıyla süpervizör endpoint'i çağırma	Her endpoint'te rol kontrolü (403 + audit log)
IDOR	Kayıt ID'sini değiştirerek başkasının verisini görme	Sunucu tarafında "bu kayıt gerçekten bu kullanıcıya mı ait" kontrolü
Token manipülasyonu	Süresi dolmuş/değiştirilmiş JWT	İmza doğrulama + süre kontrolü Gateway'de merkezi yapılır
Refresh token replay	Geçersiz kılınmış refresh token'ın tekrar kullanımı	Token rotation + theft tespitinde tüm oturumları sonlandırma
XSS	Metin alanına script etiketi	Input sanitization, mobil tarafta render edilen metnin escape edilmesi
Brute-force	Ardışık hızlı giriş denemeleri	Rate limiting + hesap kilitleme (5 deneme / 15 dk)

BÖLÜM 13

Sözlük

Terim	Tanım
Container	Bir uygulamayı ve bağımlılıklarını izole biçimde çalıştıran, host kernelini paylaşan hafif çalışma birimi.
Image	Dockerfile'ın derlenmiş, değişmez hâli; container'ların üretildiği kalıp.
Dockerfile	Bir image'ın nasıl oluşturulacağını adım adım anlatan tarif dosyası.
Docker Compose	Birden fazla container'ı tek bir YAML dosyasıyla tanımlayıp tek komutla ayağa kaldıran araç.
Volume	Container yeniden başlasa/silirse bile verinin (örn. veritabanı dosyaları) kalıcı olmasını sağlayan depolama alanı.
Mikroservis	Kendi veritabanına ve deploy döngüsüne sahip, bağımsız çalışabilen küçük servis birimi.
API Gateway	Dış dünyaya tek giriş noktası sunan, routing ve merkezi kimlik doğrulama yapan bileşen.
JWT (JSON Web Token)	Kullanıcı kimliğini ve yetkisini taşıyan, imzalı, kendi kendini doğrulayan token formatı.
Refresh Token	Access token süresi dolduğunda yenisini almak için kullanılan, daha uzun ömürlü token.
Token Rotation	Her refresh işleminde yeni refresh token üretilip eskisinin geçersiz kılınması; çalınmış token'ları tespit etmeye yarar.
Event-Driven Mimari	Servislerin birbirini doğrudan çağırmak yerine olay (event) yayınlayıp dinleyerek haberleştiği tasarım.
Message Queue	Event'lerin üretici ile tüketici arasında geçici olarak tutulduğu, asenkron iletişimi mümkün kılan altyapı (RabbitMQ, Kafka, Redis pub/sub).
Circuit Breaker	Bir bağımlılık (örn. AI Service) sürekli hata verdiğinde çağrılarını geçici olarak kesip sistemi korumaya alan desen.
State Machine	Bir varlığın (örn. optimizasyon vakası) hangi durumdan hangi duruma geçebileceğini kurallarla sınırlayan model.
SLA (Service Level Agreement)	Bir işin tamamlanması gereken azami süre taahhüdü; aşımında görsel/otomatik uyarı tetiklenir.
IDOR (Insecure Direct Object Reference)	Bir kaydın ID'sini değiştirerek başka bir kullanıcının verisine yetkisiz erişim açığı.
Rate Limiting	Belirli sürede yapılabilecek istek sayısını sınırlayarak brute-force/aşırı yük saldırılarını engelleme.

Terim	Tanım
Database-per-Service	Her mikroservisin kendi, diğerlerine kapalı veritabanına sahip olması prensibi.

Kapanış

Bu rehberdeki kavramları kafanda oturtuktan sonra, ilk günlerde (Gün 1-3) yapacağın iş aslında bu dokümanın 4. bölümündeki 9 parçanın iskeletini boş hâlde ayağa kaldırmak: her servis "merhaba dünya" seviyesinde bir health-check endpoint döner, Gateway onları doğru route eder, docker compose up hepsini tek seferde başlatır. İş mantığı (JWT, state machine, AI skorlama, event akışı) ondan sonra bu iskeletin üstüne biner.